

Helix Thready — SDK Skeleton Materials (layout + publish flow)

Table of contents

1. The two-artifact pattern

2. Per-language SDK repo layout

3. What is generated vs hand-written

4. The gate suite (no server CI)

5. Publish flow

6. How these skeletons map to the recipes

7. Open items

Rev	Date	Author	Change
1	2026-07-22	swarm (development/materials)	Initial: per-language SDK layout + publish flow accompanying the Go/TS wrapper skeletons
2	2026-07-22	swarm (docs export)	Fixed inline mermaid syntax so diagram renders

These are the **development materials** that accompany the two client skeletons in this directory (`go/client.go` , `ts/client.ts`). They are the concrete, on-disk companion to the **decision** doc `api/sdk-strategy.md` and the **usage** doc `api/sdk-examples.md` : the strategy fixes *how the 11 SDKs are produced*, the examples fix *how a caller uses them*, and this README fixes *how one SDK’s source tree is laid out and published*.

Anti-bluff status `[GAP: #11]` `[GAP: #18]` . Everything here is a **SKELETON**. The two `client.*` files are the hand-written thin layer only; the **generated core is not included** and the transport bodies are stubbed with `TODO` s that throw. Nothing here compiles into a working SDK, and the existing TS client scaffolds (6) + `UI-Components-KMP` have **no CI and no deep audit** — so a Thready SDK **must** ship its own gate suite (below) before any publish. Do not treat a skeleton as a working module.

Provenance tags per CONVENTIONS.md: `[CONSTITUTION $x]` , `[IN-HOUSE: module]` , `[RESEARCH]` , `[DEFAULT — adjustable]` , `[BUILD-NEW]` , `[GAP: id]` , `[OPEN: ...]` , `[VERIFIED]` .

Table of contents

1. The two-artifact pattern

2. Per-language SDK repo layout
3. What is generated vs hand-written
4. The gate suite (no server CI)
5. Publish flow
6. How these skeletons map to the recipes
7. Open items

1. The two-artifact pattern

Thready reuses the **mature** `helix_proto` **pattern** `[IN-HOUSE: helix_proto]` `[VERIFIED]` (`sdk-strategy.md` §1). Two schema-first contracts in the decoupled `helix_thready_proto` repo are the single source of truth:

- **OpenAPI 3.1** (`openapi/thready.v1.yaml`) → the **REST** surface → `openapi-generator` to TypeScript (`typescript-fetch`), Python, JVM, Swift, C++, C#, Ruby, PHP, Dart.
- **Protobuf** (`proto/helix/thready/v1/*.proto`) → the **event/DTO** plane and streaming RPC → `buf generate` to Go (+ Connect) and Rust (+ tonic), and Dart proto messages.

Over each **generated core** sits a **thin, hand-written idiomatic layer** (auth, retry/back-off, pagination iterators, event helpers, typed errors) — identical semantics in every language. The skeletons in this directory are that layer for Go and TypeScript.

2. Per-language SDK repo layout

Each published SDK is its own decoupled repo `[CONSTITUTION §11.4.28]`. The generated core lives under `gen/` and is **never hand-edited**; the thin layer lives beside it. Illustrative (`[DEFAULT – adjustable]`) layouts:

```

helix-thready-go/                                # Go (Critical) — buf core + REST-where-needed
├─ go.mod                                         # module github.com/helix-development/helix-thready-go
├─ gen/go/                                        # GENERATED (buf: protocolbuffers/go + connectrpc/go) —
do not edit
├─ client.go                                     # thin layer (this dir's go/client.go): Config, Auth,
services
├─ auth.go retry.go errors.go                   # thin layer split across files
├─ events.go                                     # Subscribe(): WS / Connect stream, reconnect, sticky
reconcile
├─ example_test.go                              # runnable examples (the 7 recipes)
└─ Makefile                                     # lint · generate · check-no-handwritten · test · round-
trip

helix-thready-ts/                                # TypeScript (High) — openapi-generator typescript-fetch
├─ package.json                                  # name: @helix-thready/sdk
├─ src/gen/                                      # GENERATED (openapi-generator) — do not edit
├─ src/client.ts                                # thin layer (this dir's ts/client.ts): ThreadyClient +
services
├─ src/errors.ts src/events.ts                  # ThreadyError, Subscription
├─ test/roundtrip.test.ts                       # round-trip against a stub server incl. negative-
control 401
└─ tsconfig.json                               # `tsc --noEmit` gate

```

The same shape repeats per language (`src/gen` or `gen/` core + a thin layer + a gate file): Python (`helix_thready/` + `aio/`), Kotlin/JVM (`com.helix.thready` , Kotlin-first `[OPEN: sdk-2]`), Swift (SwiftPM), Rust (`prost` + `tonic` core), C#, Ruby, PHP, and **Zig** — the one exception, which is **hand-written over the C ABI / REST** with no generated core `[OPEN: api-3]` .

3. What is generated vs hand-written

Concern	Where	Edited by hand?
Wire types / DTOs / enums	<code>gen/</code> (buf or openapi-generator)	No — regenerated from the contract; <code>check-no-handwritten</code> fails on any edit
Transport / (de)serialization	<code>gen/</code>	No
Auth injection (API key / JWT refresh)	thin layer (<code>client.go</code> / <code>client.ts</code>)	Yes
Retry on retryable codes + back-off	thin layer	Yes
Cursor pagination iterator	thin layer	Yes
Event subscription (reconnect, sticky)	thin layer	Yes
Typed error mapping	thin layer	Yes
Unknown-tolerant enums	generator config + thin layer	config only

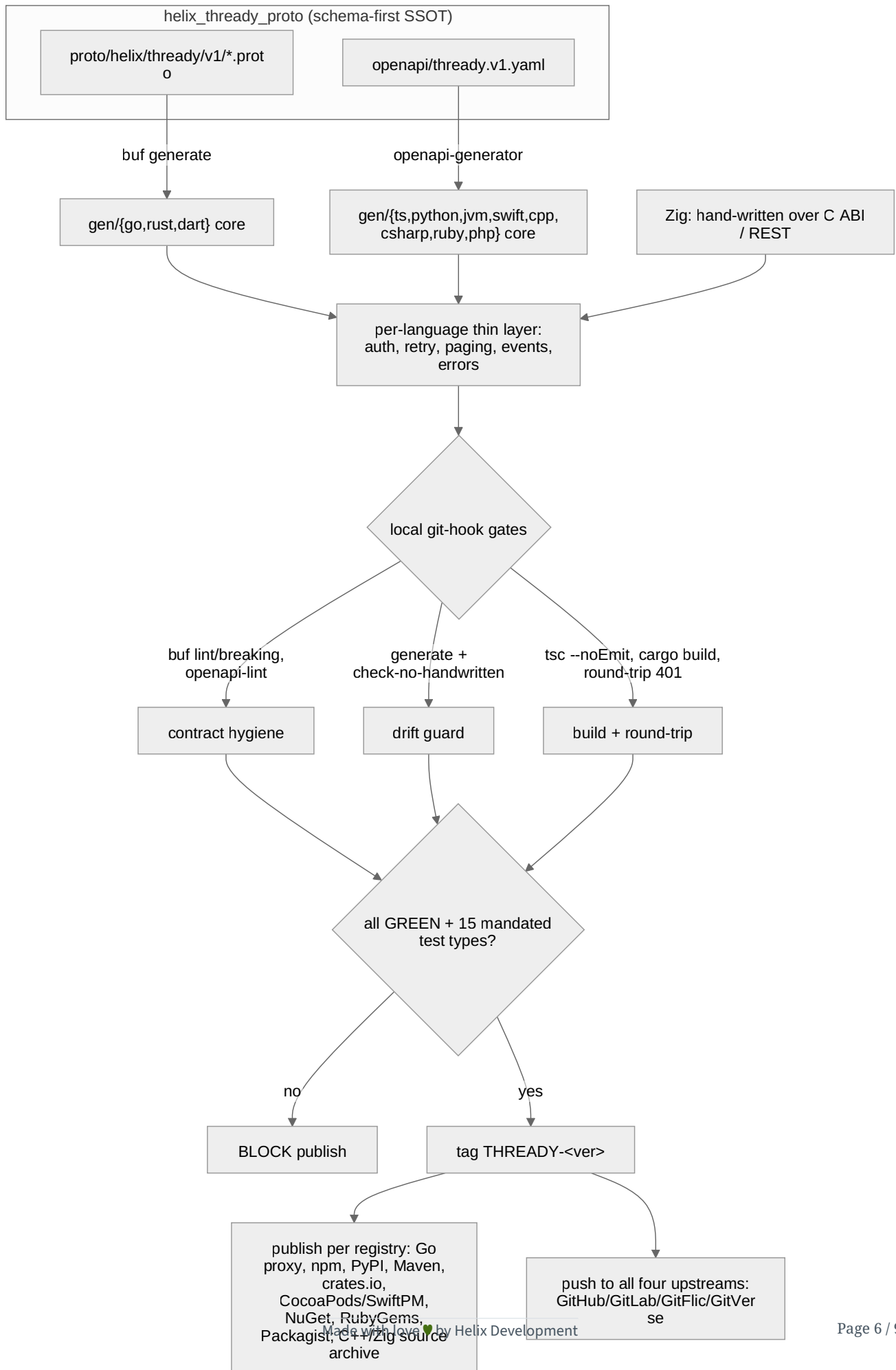
This split is enforced, not conventional: hand-editing a `gen/` file is caught by the drift gate (§4), so ergonomics can only ever live in the thin layer (sdk-strategy.md §6).

4. The gate suite (no server CI)

Server-side CI is forbidden [CONSTITUTION §11.4.156], so the gates run as **local git-hooks**, reusing helix_proto's Makefile targets verbatim [VERIFIED] (sdk-strategy.md §6):

- `buf lint` + `buf breaking` (proto) and `openapi-lint` (OpenAPI) — contract hygiene + the breaking-change gate (versioning.md).
- `generate` then `check-no-handwritten` — regenerate and assert the `gen/` cores were not hand-edited.
- `rust-build` (`cargo`), `tsc --noEmit` (TS), and a **round-trip test** driving the generated client against a real stub server **including a negative-control 401** (contract-tests.md §full-automation).
- The **15 mandated test types** [CONSTITUTION §11.4.27] for the SDK before any registry publish.

5. Publish flow



Rendered PNG/SVG exported via Docs Chain (§11.4.65). Source: ../diagrams/sdk-publish-flow.mmd.

Explanation (for readers/models that cannot see the diagram). The flow starts at the two schema-first contracts in `helix_thready_proto`: the OpenAPI document and the Protobuf definitions.

`buf generate` compiles the Protobuf into the Go/Rust/Dart cores under `gen/`, while `openapi-generator` compiles the OpenAPI into the REST-language cores (TypeScript, Python, JVM, Swift, C++, C#, Ruby, PHP). Both generated cores feed the **per-language thin layer** that adds auth, retry, pagination, events and typed errors; **Zig** joins the thin layer directly because it has no generated core and is hand-written over the C ABI / REST (`[OPEN: api-3]`).

Every wrapped SDK then passes through the **local git-hook gates** — there is no server CI (`[CONSTITUTION §11.4.156]`). Three gate groups run: contract hygiene (`buf lint` / `buf breaking` , `openapi-lint`), the **drift guard** (`generate` + `check-no-handwritten` , which regenerates the core and asserts it was never hand-edited), and build + round-trip (`tsc --noEmit` , `cargo build` , and the round-trip test that drives the generated client against a stub server including a negative-control 401). Only when **all gates are GREEN and the 15 mandated test types pass** (`[CONSTITUTION §11.4.27]`) does publish proceed; otherwise it is **blocked** — a red gate never ships. On success the release is cut on a `THREADY-<version>` tag (versioned to the contract's MAJOR.MINOR, versioning.md §7), which fans out two ways: **publish per language registry** (Go module proxy, npm `@helix-thready/sdk` , PyPI, Maven Central, crates.io, CocoaPods/SwiftPM, NuGet, RubyGems, Packagist; C++/Zig via source + release archive) and **push to all four upstreams** (GitHub/GitLab/GitFlic/GitVerse, (`[CONSTITUTION §11.4.151/§2.1]`)). Each SDK carries a full README + quickstart + reference generated md→HTML/PDF via Docs Chain (`[CONSTITUTION §11.4.65]`).

6. How these skeletons map to the recipes

Both `client.go` and `client.ts` implement the same subset of the seven recipes from `sdk-examples.md §1` with **identical semantics** (the whole point of the thin layer). The subset shown in the skeletons:

Recipe	Go (<code>go/client.go</code>)	TS (<code>ts/client.ts</code>)
R1 construct + authenticate	<code>New(Config{Auth: APIKey/JWT})</code>	<code>new ThreadyClient({ auth })</code>
R2 cursor iterator (the example call)	<code>Posts.List(...).Next().Post().Err()</code>	<code>for await (... of posts.list(...))</code>
R4 search + fail-loud embedder guard	<code>Search.Query</code> → <code>res.Embedder != "hash"</code>	<code>search.query</code> → <code>res.embedder !== "hash"</code>
R5 events subscribe (reconnect + sticky)	<code>Events.Subscribe(...)</code> → <code>sub.C</code> / <code>sub.Ack</code>	<code>events.subscribe(...).on(type, cb)</code>
R6 typed error + retry policy	<code>*thready.Error</code> , <code>Code*</code> , <code>DefaultRetry</code>	<code>ThreadyError</code> , <code>Code</code> , <code>DEFAULT_RETRY</code>
R7 transparent JWT refresh	<code>JWT(...)</code> + <code>OnTokenRotated</code>	<code>{ accessToken, refreshToken }</code> + <code>on("tokenRotated")</code>

R3 (idempotent async trigger + poll) is intentionally omitted from the skeletons to keep them to “one example call + the events subscription” per the materials scope; the full six-language R3 is in `sdk-examples.md` §3–§8. The **behaviour** (which codes retry, cursor transparency, event reconnect) is contract-fixed and identical across languages; only the package/type names are `[DEFAULT – adjustable]` and finalised at first publish per registry (`sdk-strategy.md` §7).

7. Open items

- `[OPEN: mat-sdk-1]` The generated cores (`gen/`) are **not** vendored into these materials — the skeletons reference them but the transport bodies are stubbed `TODO`s. Wiring them is tracked with the SDK build-out `[GAP: #11]`; until then nothing here is a working SDK.
- `[OPEN: sdk-1]` Event transport per language (Connect streaming vs WS/SSE) is settled in `event-bus-contract.md` §11: Go/Rust may use Connect streaming; REST-generated languages use WS/SSE. The Go skeleton notes both.
- `[OPEN: sdk-2]` One JVM artifact (Kotlin-first) vs per-language artifacts — confirm with client teams before the JVM SDK repo is cut.
- `[OPEN: api-3]` Zig has no first-class generator; its SDK is hand-written over the C ABI / REST.
- `[GAP: 7.3 Security-KMP]` The **mobile** (Android/iOS) builds MUST NOT ship until native KeyStore/Keychain replaces the in-memory secure-storage stub — server-side JVM/Swift use is unaffected (`sdk-strategy.md` §8, `authn-authz.md` §10).

Made with love ♥ by Helix Development.